

CSE - 4102

Computer Graphics and Image Processing Laboratory

Project Report

Arrakis

Harvester Down

An Interactive Procedural Desert Rescue Game

Created by:

Asique Ehetasamul Haque

Roll: **2107096**

Year: 4th

Semester: 1st

Group: B2

Department of Computer Science and Engineering,
Khulna University of Engineering and Technology

Contents

1	Description	1
1.1	Objectives	1
1.2	Mission in Plain Language	1
1.3	Implemented Feature Summary	1
1.4	Reading Guide and Scope	2
1.5	Essential Graphics Terms	3
2	Objects and Procedural Geometry	4
2.1	Actual Scene Screenshots	4
2.2	Mesh Data and Model Assembly	6
2.3	Reusable Shapes and Curved Equations	6
2.4	Desert Height Field	8
2.5	Ornithopter: Controllable Vehicle	8
2.6	Spice Harvester: Autonomous Machinery	9
2.7	Spice Tanks	9
2.8	Rock Formations	9
2.9	Rescue Base, Markers, Workers and Winch	10
2.10	Sandworm: Detailed Organic Object	10
3	Motion, Control and Dynamic Scenes	14
3.1	Time and Simulation Design	14
3.2	Player-Controlled Aircraft	14
3.3	Wing and Machinery Animation	15
3.4	Harvester Route and Terrain Alignment	16
3.5	Worker Evacuation and Rescue States	16
3.6	Worm Pursuit, Attack and Scene Escalation	17
3.7	Camera Movement	18
3.8	Particles as Dynamic Environmental Objects	18
4	Rendering, Appearance and Additional Features	20
4.1	Forward Rendering Pipeline	20
4.2	Material Design	20
4.3	Lighting Equation	21
4.4	Procedural Textures and Surface Detail	22
4.5	Shadows: Depth Map, Bias and Filtering	22
4.6	GPU Terrain Collapse During the Attack	23

4.7	Procedural Sky and Sun	23
4.8	Tone Mapping and Distance Fog	24
4.9	Instanced Transparent Dust	24
4.10	Inspection and Robustness Features	24
5	Complete Project Pipeline	26
5.1	Source Organization and Responsibilities	26
5.2	From Source to a Running Frame	26
5.3	Complete User Controls	27
5.4	HUD and Radar Implementation	28
5.5	Audio Pipeline and Event Design	28
5.6	Build, Dependencies and Distribution	29
5.7	Testing and Observed Results	30
5.8	Documentation and Report Production	31
6	Implementation References	32

1 Description

Arrakis: Harvester Down is an interactive, real-time desert rescue game implemented in C++17 with OpenGL 3.3. The player pilots an ornithopter over a procedurally generated desert, rescues workers escaping a moving spice harvester, and delivers them to a safe landing pad before a pursuing sandworm reaches the harvester. The setting is inspired by the desert world of *Dune*; the meshes, rendering, animation and mission rules are implemented in the project rather than loaded from a game engine.

The scene is not merely a static model viewer. It combines a controllable aircraft, autonomous vehicles and characters, an animated sandworm, dust particles, a changing rescue mission, camera control, on-screen instrumentation and event-driven audio. All visible geometry is generated in code. Surface detail is also procedural: no external 3D models or image textures are loaded. Runtime file assets are the sound effects.

1.1 Objectives

- Construct recognizable objects from triangle meshes and reusable parametric shapes.
- Explain curved surfaces and complex motion with equations that correspond to the implementation.
- Provide responsive, game-like control of a dynamic object and its camera.
- Integrate autonomous motion, timed events and interaction into one dynamic scene.
- Apply lighting, shadows, procedural surface detail, atmospheric effects and transparent particles.
- Document the entire path from source code and assets to an interactive rendered frame and verifiable demonstration.

1.2 Mission in Plain Language

At the start of a mission, 36 workers are inside the harvester. Small groups evacuate, run away from it and wait at amber rescue beacons. The aircraft can carry at most eight workers. The player must fly low and steadily, hold the winch/brake control near a group, and then unload at the cyan base. A worker is counted as *secured* only after delivery; merely boarding the aircraft does not increase the final rescue score.

The first pursuit lasts 210 seconds. Each later mission reduces this duration by 10 seconds, down to a minimum of 150 seconds. A worm attack ends the harvester phase, followed by a 60-second final extraction window. Remaining exposed workers can still be rescued during that window. At debrief, the project reports the outcome and allows retry or the next mission. The best score is retained during the running application, not saved permanently to a file.

1.3 Implemented Feature Summary

Area	Implemented features
Objects	Dune terrain, ornithopter with articulated wings, multipart spice harvester, three spice tanks, rock formations, landing pad and lights, evacuation markers, worker characters, winch line, detailed sandworm and underground wake.

Area	Implemented features
Curved geometry	Ellipsoids, cylinders and discs, annular rings, a multi-frequency height field, an organic segmented worm body, a recessed mouth, concentric lips and curved teeth.
Motion and interaction	Terrain-following aircraft with acceleration and damping, strafing and boost; autonomous harvester route; evacuation and running workers; winching and delivery; worm pursuit, rise, attack and retreat; wing and machinery animation.
Dynamic scenes	Briefing, active rescue, pursuit escalation, evacuation, worm breach, harvester consumption, final extraction, pause, debrief, retry and increasing mission difficulty.
Graphics	Indexed triangle meshes, transformations, GLSL lighting and procedural materials, directional shadow mapping with filtering, generated sky and sun, distance fog, GPU terrain collapse, instanced transparent dust, wireframe inspection.
Interface	Bitmap-font HUD, world radar, crew and cargo counters, timers, altitude/speed/FPS, boost and danger bars, contextual messages, menu/pause/debrief overlays, three camera presets, fullscreen and focus handling.
Audio	Eight runtime sound effects, looping vehicle/worm ambience, changing pitch and volume, event sounds, mute, pause silencing, and graceful failure when audio is unavailable.
Engineering	CMake/Visual Studio builds, dependency discovery, audio copying, deterministic mission tests, graphical smoke tests with BMP capture, and a reproducible documentation tool.

1.4 Reading Guide and Scope

Sections 2–4 explain objects, motion and rendering with equations. Section 5 connects them into the complete application pipeline, including controls, audio, build and verification. Section 6 maps the assessment requirements to evidence and supplies the exact ten-slide structure and a 120-second demonstration plan.

This report describes the **current source implementation**, not unimplemented proposal ideas. The screenshots are actual application frames captured from the current Release build. The presentation outline and video timeline are preparation instructions: this report does not claim that a slide deck or a two-minute recording has already been produced. Where a technical term or marking weight was not defined in the assignment, that uncertainty is stated rather than guessed.

1.5 Essential Graphics Terms

Term	Meaning in this report
Mesh	Vertices joined into triangles to approximate an object's surface.
Normal	A direction perpendicular to the surface, used to calculate lighting.
Shader	A small GPU program that transforms vertices or colours individual fragments.
Fragment	A candidate contribution to a screen pixel produced while rasterizing a triangle.
Procedural	Generated by code and mathematical rules instead of imported artwork.
UV	Two surface coordinates often used for textures; present in the meshes even when the appearance uses world coordinates.
Framebuffer	The rendering destination; either the window or an off-screen target such as the shadow map.
Instancing	Drawing many copies of one mesh with different per-copy data in one draw call.

Implementation reference:

README.md; src/arrakis.cpp; src/arrakis_game.h; src/arrakis_worm.h; src/arrakis_hud.h; src/arrakis_audio.h

2 Objects and Procedural Geometry

2.1 Actual Scene Screenshots

These figures come from the current application, not reference artwork. They show the shared scene context; the inventory and equations below explain details that are too small to inspect in a wide view. A still image illustrates appearance, while the presentation/video should demonstrate movement and interaction.

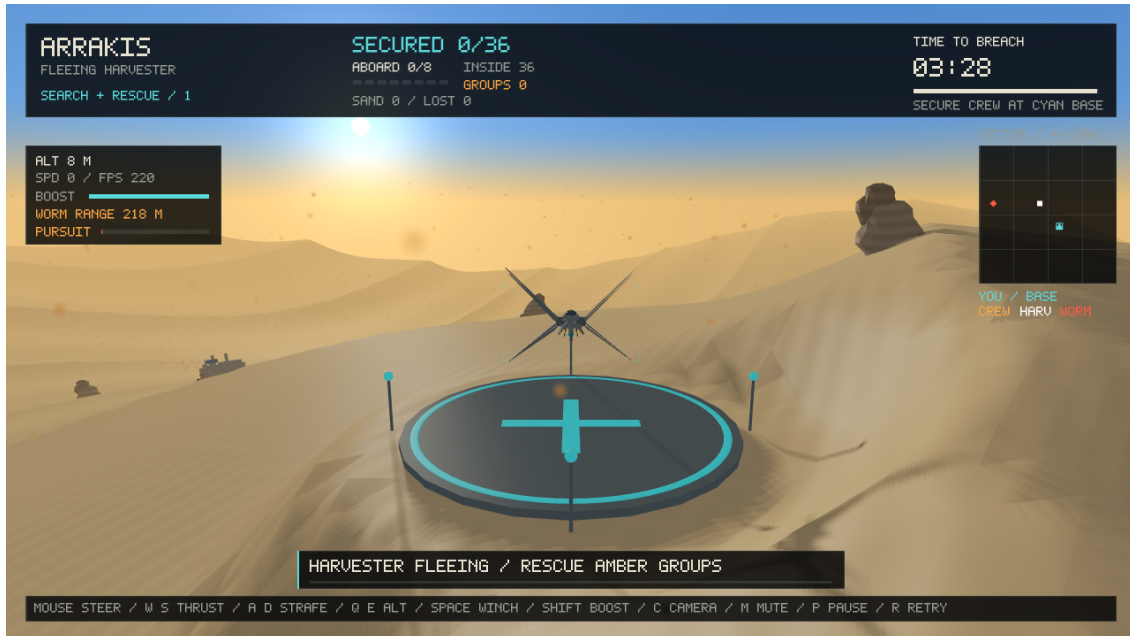


Figure 1: Initial rescue scene: player aircraft over the cyan landing base, dune terrain, rocks, distant harvester, atmospheric dust and mission HUD. Captured with --smoke-test.



Figure 2: Worm breach: open mouth with dense curved teeth, player aircraft, crew in the distance, dust and long directional shadows. This smoke setup advances to breach+7 seconds before capture.

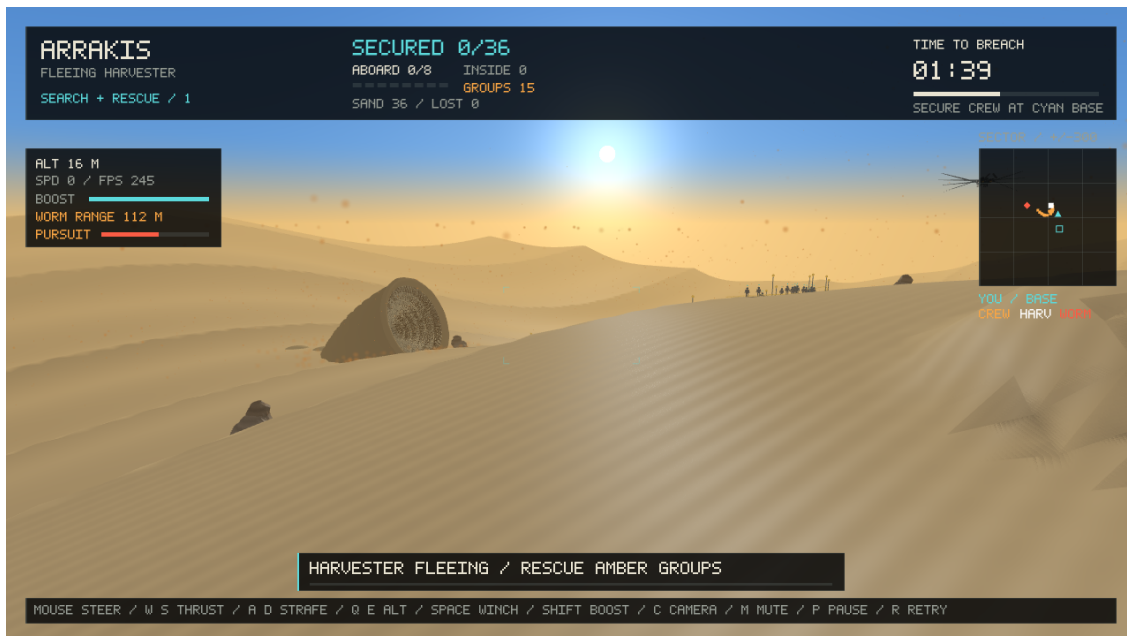


Figure 3: Pursuit scene after evacuation: sandworm at left, amber worker groups on the ridge, aircraft at right and radar/danger feedback. The setup advances to 110 mission seconds, so crew are already outside.



Figure 4: Tactical camera in the mouse-control smoke test: aircraft, cyan base, terrain and shadows viewed from above. The test changes steering and camera presets over 120 frames.

2.2 Mesh Data and Model Assembly

Each scene vertex contains a 3D position, 3D normal and 2D UV coordinate. A mesh owns a vertex array object (VAO), vertex buffer (VBO), element/index buffer (EBO) and index count. Triangles are drawn with unsigned integer indices. Most generated geometry is uploaded once with `GL_STATIC_DRAW`; the deforming worm body uses a dynamic buffer.

Complex objects reuse a small set of shapes. For a part with local translation, rotation and scale, the assembly helper forms

$$M = M_{\text{root}} T R_y R_x R_z S. \quad (1)$$

With column vectors, scale acts first, followed by local rotations, translation and the parent's transform. Parent matrices attach wings, machinery and decorations to their respective vehicles. The same object traversal is used in the shadow and visible passes, preventing separate manually maintained shadow models.

Implementation reference:

`src/arrakis.cpp:261--309,980--1020; src/arrakis_worm.h:274--279`

2.3 Reusable Shapes and Curved Equations

Shape	Construction	Runtime topology
Cube	Six independent faces, side length 1, flat normals; used for panels, tracks, fins and markings.	24 vertices, 12 triangles.
Cylinder	Radius 0.5, height 1 along local Y , 24 slices, separately capped ends.	102 vertices, 96 triangles.
UV sphere	Radius 0.5, 16 latitude bands, 24 longitude sectors; smooth radial normals.	425 vertices, 768 submitted triangles, including degenerate pole triangles.
Wing	Nine four-point sections forming a tapered blade profile; root/tip are not capped.	36 vertices, 64 triangles per blade.
Torus	80 major-circle intervals, eight tube intervals, major radius 1, tube radius 0.025.	729 vertices, 1,280 triangles.
Eroded rock	Perturbed sphere-like sample grid, discarded zero-area triangles, per-face normals.	182 nondegenerate triangles with independent face vertices.
Terrain	260×260 cells, two triangles per cell, finite-difference normals.	68,121 vertices, 135,200 triangles.
Particle quad	Unit square in local XY , oriented toward the camera.	Four vertices, two triangles per instance.

2.3.1 Sphere and ellipsoid

For latitude $\theta \in [0, \pi]$ and longitude $\phi \in [0, 2\pi]$,

$$\mathbf{n} = (\sin \theta \cos \phi, \cos \theta, \sin \theta \sin \phi), \quad \mathbf{p} = 0.5\mathbf{n}. \quad (2)$$

The samples use $\theta = \pi i/16$, $\phi = 2\pi j/24$ and UV $(j/24, i/16)$. Scaling a sphere by (s_x, s_y, s_z) gives the ellipsoid

$$\frac{x^2}{(s_x/2)^2} + \frac{y^2}{(s_y/2)^2} + \frac{z^2}{(s_z/2)^2} = 1. \quad (3)$$

Ellipsoids create the aircraft hull/canopy, worker torso/helmet/visor, tank ends and worm wake. The equation describes each local unrotated instance; its model matrix then places and orients it in the scene.

2.3.2 Cylinder

For $\theta_i = 2\pi i/24$, the two side rings are

$$\mathbf{p}_i^\pm = (0.5 \cos \theta_i, \pm 0.5, 0.5 \sin \theta_i), \quad \mathbf{n}_i = (\cos \theta_i, 0, \sin \theta_i). \quad (4)$$

Caps use separate vertices and upward/downward normals for hard edges. Under nonuniform scale, the side satisfies

$$\frac{x^2}{(s_x/2)^2} + \frac{z^2}{(s_z/2)^2} = 1, \quad -s_y/2 \leq y \leq s_y/2. \quad (5)$$

The primitive is reused for tanks, engine pods, wheels, limbs, poles, pad and winch line. Objects called needles, stingers or separator funnels are still scaled cylinders, not separately generated cones or hollow pipes.

2.3.3 Torus and annular markers

For major angle a and tube angle b ,

$$\mathbf{p}(a, b) = ((1 + 0.025 \cos b) \cos a, 0.025 \sin b, (1 + 0.025 \cos b) \sin a). \quad (6)$$

This is a thin 3D ring tube, not a flat disc. The landing-pad ring scales horizontally by 11 and evacuation rings by 4.5. Its UV coordinates follow the two periodic angles.

2.3.4 Wing blade profile

At section $t = i/8$, chord $c = 1 - 0.78t$ and thickness $q = 0.08 - 0.065t$. Its four points are

$$P_{\text{lead}} = (t, 0, -0.45c), \quad P_{\text{top}} = (t, 0.5q, -0.1c), \quad (7)$$

$$P_{\text{trail}} = (t, 0, 0.55c), \quad P_{\text{bottom}} = (t, -0.5q, -0.1c). \quad (8)$$

Neighbouring sections are joined around the profile. Chord and thickness shrink toward the tip, producing a long, tapered mechanical blade. The normal directions are profile approximations, not exact analytical normals of a smooth airfoil.

Implementation reference:

src/arrakis.cpp:312--499,543--603

2.4 Desert Height Field

2.4.1 The current terrain equation

The terrain combines broad waves, folded ridges and smaller undulations:

$$\begin{aligned} H_0(x, z) = & 28 \sin(0.009x + 0.004z) + 19 \cos(0.004x - 0.012z) \\ & + 5[1 - |\sin(0.028x + 0.016z)|]^2 \\ & + 1.8 \sin(0.065x + 0.038z) - 4. \end{aligned} \quad (9)$$

The first two terms produce large diagonal swells; the folded sine creates repeated sharper ridges; the final sine adds smaller shape variation. These are mathematical dunes, not a physical simulation of sand erosion or truly asymmetric windward/leeward dune formation.

The base at (55, 55) is flattened by blending to $h_c = H_0(55, 55)$. For $d = \sqrt{(x - 55)^2 + (z - 55)^2}$,

$$b = 1 - \text{smoothstep}(15, 25, d), \quad H(x, z) = (1 - b)H_0(x, z) + bh_c. \quad (10)$$

The inner radius 15 is flat, the exterior beyond radius 25 remains unchanged, and the intervening band transitions smoothly. In these equations, `smoothstep` maps its normalized interval with $3u^2 - 2u^3$, after clamping u to $[0, 1]$.

2.4.2 Sampling and normals

The current application creates a 900-by-900 patch with 260 cells in each direction:

$$x_i = -450 + i \frac{900}{260}, \quad z_j = -450 + j \frac{900}{260}. \quad (11)$$

The spacing is about 3.462 units. Every grid cell becomes two triangles. Heights are sampled from H and normals are computed by central finite differences with $\epsilon = 0.4$:

$$\mathbf{n} = \text{normalize} \left(-[H(x + \epsilon, z) - H(x - \epsilon, z)], \ 2\epsilon, \ -[H(x, z + \epsilon) - H(x, z - \epsilon)] \right). \quad (12)$$

This includes the flat-pad blend and gives smoother lighting than independent triangle face normals. UV coordinates are $(0.05x, 0.05z)$, although visible sand patterns use world position instead of an imported texture.

The terrain is static and finite, without streaming, adaptive tessellation or levels of detail. CPU height queries evaluate the continuous equation while the displayed surface interpolates triangle vertices. The current amplitudes 28/19 and grid 260 differ from older project comments/documentation.

Implementation reference:

`src/arrakis.cpp:108--144, 502--539, 1789`

2.5 Ornithopter: Controllable Vehicle

The aircraft has 24 fixed primitive parts plus eight articulated wing meshes. A stretched sphere forms the main hull (scale 1.3, 0.85, 3.2); another forms the dark canopy (0.85, 0.65, 1.6). Cylinders supply the nose, sensor needle, paired dorsal intakes, front/rear wing gimbals, twin turbine bodies, twin nozzles, two exhaust flames, two tail-boom segments and stinger. Cubes form the canopy spine, two V-tail fins and two landing skids.

The four wings on each side attach at front/rear, upper/lower pivots. Each has a 5.2-unit signed span scale, a fixed sweep/dihedral and time-varying hinge/twist rotations. Negative scale mirrors the other side; face culling is disabled for the blades so both sides remain visible. Flap motion and boost changes are detailed in Section 3.

Hull, dark mechanical parts, canopy, wings and nozzles have distinct diffuse/specular/shininess values. The canopy is glossy but opaque: it is not transparent or refractive glass. The visible exhaust is emissive cylinder geometry, not a fluid flame. There is no cockpit interior, pilot mesh, retracting landing gear or weapons subsystem.

Implementation reference:

`src/arrakis.cpp:1049--1207,1527--1534`

2.6 Spice Harvester: Autonomous Machinery

The harvester is a 92-part primitive assembly. Its large main hopper ($9.5 \times 3.8 \times 6.2$) and upper refinery deck ($8.2 \times 1.2 \times 5$) give the industrial silhouette. Four track units sit at the front/back of each side; each contains a housing, four cylindrical wheels with decorative rotating bars, and ten separately animated track-pad cubes.

Other components are the front cutter drum, side exit door, inclined ramp, emissive spice scoop, two side separators, raised bridge, glossy viewport, two emissive floodlight ornaments, twin exhaust stacks and bright vent rims. The drum and tracks rotate with travel. The root follows the route and terrain slope, then sinks and tilts during consumption.

The separator shapes are straight capped cylinders rather than true funnel profiles. Floodlight ornaments glow visually but do not create spotlights illuminating surrounding sand. There are no separately modeled cutter teeth or dedicated black stack smoke; machinery dust comes from the shared particle system. The harvester is autonomous, not another player-controlled vehicle.

Implementation reference:

`src/arrakis.cpp:1213--1313,1536--1555`

2.7 Spice Tanks

Three stationary tanks are initialized at $XZ = (-12, -26), (-19, -32), (-26, -23)$ with yaws $20^\circ, -15^\circ, 10^\circ$. Each is drawn at the terrain height with uniform scale 1. They are not player-deployable objects.

Each tank contains 16 parts: two skids, two cross supports, four inclined stanchions, one horizontal vessel cylinder, two overlapping ellipsoidal end heads, two reinforcing bands, a top hatch, an emissive valve and a vertical indicator strip. The vessel diameter is 2.2 and length is 3.2; its end ellipsoids have diameters (2.2, 2.2, 1.3) and centres at local $z = \pm 1.6, y = 1.55$. One end satisfies

$$\frac{x^2}{1.1^2} + \frac{(y - 1.55)^2}{1.1^2} + \frac{(z - z_c)^2}{0.65^2} = 1. \quad (13)$$

The exterior silhouette resembles a pressure vessel through overlap; no cut-hemisphere/cap boolean operation is used. The gauge/valve emission is constant, not pulsing or a simulated spice fill level. Tanks follow ground position but do not tilt to its normal, and disappear with the harvester attack phase.

Implementation reference:

`src/arrakis_game.h:155--158; src/arrakis.cpp:1320--1382,1557--1567`

2.8 Rock Formations

Eight sites contain four differently scaled/rotated rock pieces each, for 32 rock-mesh instances. The sites are (45, -65), (-55, 40), (90, 30), (-90, -70), (15, 95), (-25, -115), (120, -25) and (-130, 85). Their roots use terrain height minus 0.5 and successive yaw offsets, reducing obvious repeated alignment.

The rock mesh starts with latitude $a = \pi l/8$ and longitude $b = 2\pi k/13$, then perturbs its horizontal

radius:

$$e(l, k) = 0.83 + 0.13 \sin(7.1k + 2.3l) + 0.08 \cos(5.4l), \quad (14)$$

$$\mathbf{p}(l, k) = \frac{1}{2}(\sin a \cos b e, \cos a, \sin a \sin b e). \quad (15)$$

Each nondegenerate triangle gets an outward flat normal from its edge cross product. Light/dark strata materials add fragment-shader bands and grain. The perturbation is deterministic; it is not simulated erosion. Since the sample perturbation is index-dependent, the duplicated angular seam is not perfectly welded mathematically. Nearby rock sites also raise aircraft clearance, but there is no exact rock-mesh collision solver.

Implementation reference:

src/arrakis_game.h:38--39; src/arrakis.cpp:573--603,1388--1406,1569--1577

2.9 Rescue Base, Markers, Workers and Winch

The base is a dark cylinder of diameter 24 and height 0.5, with a cyan radius-11 torus and two crossed strips. Four poles at radius 13 carry cyan lamp spheres. It is located at the flattened terrain centre (55, 55) and uses 12 primitive draws.

Each active evacuation group has an amber radius-4.5 torus and a pole/lamp. The ring rotates from world up to the centre terrain normal using axis $(0, 1, 0) \times \mathbf{n}$ and angle $\arccos(n_y)$. It is slope-aligned, not deformed to the terrain around its entire circumference. Poles remain vertical. A group's marker disappears when no running/waiting members remain.

A visible worker uses eight parts: torso ellipsoid, helmet/head ellipsoid, visor ellipsoid, two cylindrical arms, two cylindrical legs and a cyan locator sphere. The torso diameters are (0.55, 0.85, 0.4) and head diameters (0.43, 0.46, 0.43). Opposite arms/legs rotate to create running/idle motion. There is no skeletal rig, knee/elbow articulation or heading-oriented body root.

The winch line is a thin cyan cylinder of diameter 0.055 between the aircraft and the lifted target. Its centre is the midpoint, its length is endpoint distance, and a rotation aligns local Y with the line direction. It is straight rather than a simulated flexible cable. Gameplay conditions and scoring are detailed in Section 3.

Implementation reference:

src/arrakis.cpp:1427--1508

2.10 Sandworm: Detailed Organic Object

The worm is not a single scaled cylinder. It combines a deformable ridged body, a shaped mouth lip, recessed funnel, deep throat and 1,000 individually curved teeth combined into one mesh. Five mesh draws represent 54,230 vertices and 99,704 submitted triangles; four additional sphere instances form its surface wake.

Mesh	Sampling	Vertices	Triangles
Body	192 length intervals, 96 angular intervals	18,721	36,864
Lip	Eight profile rings, 160 angular intervals	1,288	2,240
Funnel	Fourteen profile rings	2,254	4,160
Throat	Five profile rings plus bottom fan	967	1,440
Teeth	1,000 five-sided curved teeth	31,000	55,000

2.10.1 Body centreline and surface

The neck bends in a local YZ plane. For head Q , tilt τ and axis $U = (0, \cos \tau, \sin \tau)$, the lip join is $E = Q - 1.4U$. At neck parameter $s \in [0, 1]$,

$$q = \frac{1}{2}\tau(1-s), \quad m = \frac{1}{2}\tau(1+s), \quad L_s = 64(1-s) \operatorname{sinc}(q), \quad (16)$$

$$C_{\text{neck}}(s) = E - (0, L_s \cos m, L_s \sin m), \quad (17)$$

where $\operatorname{sinc}(q) = \sin(q)/q$ with the implemented small-positive- q limit set to 1. The neck length is 64, and it becomes straight as tilt tends to zero.

The buried body joins a point 140 units beneath local terrain to the neck using a cubic Bézier curve:

$$C_{\text{buried}}(s) = (1-s)^3 P_0 + 3(1-s)^2 s P_1 + 3(1-s) s^2 P_2 + s^3 P_3. \quad (18)$$

$P_3 = C_{\text{neck}}(0)$, P_0 is the buried start, and control points $P_1 = P_0 + (0, b, 0)$, $P_2 = P_3 - (0, b, 0)$ use $b = (P_{3y} - P_{0y})/3$. The first 40% of body parameter t samples this buried curve; the remainder samples the neck.

For tangent T , let $V = (0, -T_z, T_y)$ and radial direction $R(a) = (\cos a, 0, 0) + \sin a V$. Define taper $p = 1 - \text{smoothstep}(0.85, 1, t)$ and aperture A . The radial terms are

$$r_{\text{ridge}} = 0.70[0.5 + 0.5 \cos(2\pi 32t)]^3, \quad (19)$$

$$r = \text{mix}(13.9 + r_{\text{ridge}}, 15A, 1 - p), \quad (20)$$

$$g = p[0.11 \sin(13a + 9t) + 0.07 \sin(29a - 17t)], \quad (21)$$

$$P(t, a) = C(t) + (r + g)R(a). \quad (22)$$

The ridge term gives 32 body-ring cycles, grain breaks uniformity, and taper blends the body into the mouth aperture. Normals are recomputed from longitudinal/angular positional differences, so bending and ridges affect lighting. Updated vertices stream to the dynamic body VBO. Cached identical pose inputs prevent doing the deformation twice for shadow and colour passes.

Implementation reference:

src/arrakis_worm.h:197--255, 274--279, 303--315

2.10.2 Lip, funnel and throat by surface revolution

For profile pair (r_j, y_j) and angle a , a perturbed radius generates a ring:

$$\rho_j(a) = r_j + R \sin\left(\frac{\pi j}{m-1}\right) [0.55 \sin(11a + 0.7j) + 0.30 \sin(23a - 0.9j) + 0.15 \sin(47a + 1.1j)], \quad (23)$$

$$P_j(a) = (\rho_j \cos a, y_j, \rho_j \sin a). \quad (24)$$

R is 0.16 for the lip, 0.08 for the funnel and zero for the throat. Small angular ripples produce an organic edge. Adjacent profile rings are joined by triangles, and shared normals are accumulated with the angular seam averaged.

The exact profiles below list (*radius, local axial coordinate*). Negative axial coordinates recess into the mouth.

```
Lip:
(15.0,-1.4), (15.6,-0.9), (15.9,-0.2), (15.5,0.5),
(14.8,0.8), (14.0,0.5), (13.4,-0.1), (13.1,-0.8)
Funnel:
(13.1,-0.8), (12.8,-1.3), (12.0,-2.0), (11.3,-2.6),
```

```
(10.5, -3.5), (9.4, -4.5), (8.6, -5.4), (7.7, -6.3),
(6.8, -7.3), (6.0, -8.2), (5.2, -9.2), (4.5, -10.1),
(3.6, -11.4), (2.9, -13.0)
Throat:
(2.9, -13.0), (2.6, -15.0), (2.2, -19.0),
(1.7, -24.0), (0.8, -27.0)
```

Only the deep throat bottom is closed with a radius-0.8 fan at axial coordinate -27; there is no disc across the mouth entrance. Darker interior materials strengthen the visible depth but do not replace the modeled cavity.

Implementation reference:

src/arrakis_worm.h:45--95,281--289

2.10.3 Curved teeth

Six tooth rows use counts (192, 184, 176, 168, 152, 128), radii (13.1, 11.3, 9.4, 7.7, 6.0, 4.5) and axial depths (-0.8, -2.6, -4.5, -6.3, -8.2, -10.1). Counts sum to 1,000.

For tooth k in row j , define $\sigma = 17k + 131j$, $v = 0.5 + 0.5 \sin(1.713\sigma)$ and

$$a = \frac{2\pi[k + 0.37j + 0.18 \sin \sigma]}{\text{count}_j}. \quad (25)$$

Let $R_a = (\cos a, 0, \sin a)$ and $S_a = (-\sin a, 0, \cos a)$. With row radius r_j , depth d_j ,

$$\ell = (r_j - 1.8)(0.55 + 0.17v), \quad D = 2.1 + 1.1v, \quad (26)$$

$$c = 0.20 + 0.30 \sin(0.91\sigma), \quad w = (0.075 + 0.035v)(1 - 0.06j). \quad (27)$$

The curved centreline is

$$P(t) = R_a(r_j - \ell t) + S_a(ct^2) + (0, d_j + 0.65 \sin \pi t - Dt^2, 0). \quad (28)$$

This points inward, bends sideways and drops into the mouth. Its tangent is the normalized derivative. A moving cross-section frame uses $B = \text{normalize}(T \times S_a)$ and $N = B \times T$. At five angular sides $\beta = 2\pi s/5$,

$$P_{\text{surface}} = P(t) + w(1 - t)^{0.85}[N \cos \beta + B \sin \beta]. \quad (29)$$

Six rings at $t = 0, 1/6, \dots, 5/6$ and one true apex at $t = 1$ produce 31 vertices and 55 triangles per tooth. The attachment base is open. All teeth move with the head matrix; they do not individually flap or grow. Their moving-frame normals are approximations, appropriate for the small geometry.

Implementation reference:

src/arrakis_worm.h:99--159

2.10.4 Rise, aiming, swallowing and retreat

For attack time a , define rise $r = \text{smoothstep}(0, 6, a)$, lunge $l = \text{smoothstep}(6, 12, a)$ and retreat $e = \text{smoothstep}(13.5, 18, a)$. Before breach, head height is $10 + 4 \text{smoothstep}(0, 1, \text{progress})$ above terrain. During attack it becomes

$$h = 14 + 22r - 27l + 0.25 \sin(1.6t)r(1 - l), \quad (30)$$

then blends toward -24 with e . Forward reach is $12l(1 - e)$. A terrain sample 18 units toward the heading defines target height y_T and aim angle $\text{atan2}(18 - \text{reach}, y_T - y_{\text{head}})$. Tilt is $70^\circ(1 - r) + \text{aim}l(1 - e)$, with the first term converted to radians. Aperture becomes

$$A = 1 - 0.18 \text{smoothstep}(12, 13.5, a)(1 - e). \quad (31)$$

The head parts share $T(Q)R_x(\tau)S(A, 1, A)$; a heading rotation places the local bend toward the harvester. The body deforms to match. This is authored parametric animation, not a skeletal or physics-driven creature.

Four sand-coloured ellipsoids behind the worm form the wake. Their fade is $w = 1 - \text{smoothstep}(0, 6, a)$ and their scales are $((12 - i)w, [3 + 0.35 \sin(2t - i)]w, 10w)$. They are partly embedded in terrain and fade as the worm rises. Their movement is separate from the dust emitters and terrain-collapse effect.

Implementation reference:

`src/arrakis_worm.h:168--195,294--356`

3 Motion, Control and Dynamic Scenes

3.1 Time and Simulation Design

Let Δt be the time step in seconds. Simulation positions use game-world units; the HUD labels these as metres for player readability. Angles in trigonometric functions are radians unless degrees are stated explicitly. The application clamps a long frame interval to 0.1 seconds and divides mission updates into steps no larger than approximately 1/120 second. This limits instability and makes scheduled interactions less dependent on rendering speed.

Many smooth responses use the exponential interpolation factor

$$\alpha_k = 1 - e^{-k\Delta t}, \quad x_{\text{new}} = (1 - \alpha_k)x + \alpha_k x_{\text{target}}. \quad (32)$$

Here k is a response rate: a larger value reaches the target faster. Unlike a fixed per-frame interpolation amount, this factor adjusts naturally to frame duration. It does not make every part of the entire simulation perfectly frame-rate independent; positions are still advanced in discrete steps.

Mission elapsed time, wing phase and the presentation clock are separate values. Pausing stops mission/presentation/particle advancement, but camera smoothing can still settle into position. The main application also advances the elapsed clock in debrief so the worm's final animation can finish, even though the standalone mission update no longer advances gameplay there.

Implementation reference:

src/arrakis.cpp:1855--1946; src/arrakis_game.h:168--214

3.2 Player-Controlled Aircraft

3.2.1 Heading, mouse steering and banking

For aircraft yaw θ , the horizontal forward and right vectors are

$$\mathbf{f} = (-\sin \theta, 0, -\cos \theta), \quad \mathbf{r} = \mathbf{f} \times (0, 1, 0). \quad (33)$$

Thus yaw zero faces negative Z . Forward/reverse and strafe controls are evaluated in this local basis; strafing does not force the nose to face the movement direction.

Mouse position is normalized using the logical window dimensions:

$$u_x = 2x/w - 1, \quad u_y = 2y/h - 1, \quad D(u) = \text{sign}(u) \max\left(0, \frac{|u| - 0.12}{0.88}\right). \quad (34)$$

The central dead zone occupies $\pm 6\%$ of the full width/height around the screen centre. Moving the mouse outside it steers continuously without requiring a click. Invalid or outside-window input clears the steering values.

Horizontal mouse input and left/right arrows combine into

$$s = \text{clamp}(D(u_x) + I_{\text{Right}} - I_{\text{Left}}, -1, 1), \quad q = -80s, \quad (35)$$

where q is the target yaw rate in degrees per second. With previous rate ω and $d = e^{-8\Delta t}$, the code analytically integrates the smoothed turn:

$$\theta_{\text{new}} = \text{remainder}\left(\theta + q\Delta t + \frac{(\omega - q)(1 - d)}{8}, 360\right), \quad (36)$$

$$\omega_{\text{new}} = q + (\omega - q)d. \quad (37)$$

This yields responsive but non-abrupt rotation. Returning the cursor to the centre damps turn rate; it does not snap heading back. Aircraft roll follows 0.28ω with response rate 6, producing visible banking.

Implementation reference:

src/arrakis_game.h:91--105,175--184

3.2.2 Translation, braking and boost

The input vector is

$$\mathbf{i} = \mathbf{r}(I_D - I_A) + \mathbf{f}(I_W - I_S). \quad (38)$$

Any nonzero vector is normalized, preventing diagonal motion from being faster. Target speed is 28 normally or 48 when boosting. Space reduces commanded speed to 10% and increases damping, allowing low-speed rescue hovering:

$$\mathbf{v}_d = V\hat{\mathbf{i}}, \quad \mathbf{v}_{\text{new}} = \text{mix}(\mathbf{v}, \mathbf{v}_d, 1 - e^{-k\Delta t}), \quad \mathbf{p}_{\text{new}} = \mathbf{p} + \mathbf{v}_{\text{new}}\Delta t, \quad (39)$$

where $k = 5.5$ normally and $k = 12$ with Space. Horizontal position is limited to $[-160, 160]$ in both X and Z .

Boost charge starts at 1, drains by $0.22\Delta t$ while boosting and recharges by $0.14\Delta t$ otherwise, clamped to $[0, 1]$. Boost requires movement, Left Shift and no Space. At charge ≤ 0.04 , a lock prevents immediate automatic reactivation; the player must release Shift to clear it. Boost also changes wing speed, engine-flame length and sound.

Implementation reference:

src/arrakis_game.h:183--198

3.2.3 Terrain following and obstacle clearance

Q/E changes requested altitude by $15(I_Q - I_E)\Delta t$, clamped to 4–55. Space smoothly pulls it toward 6. The target aircraft height is terrain height plus the requested clearance, modified near obstacles. At a rock distance d ,

$$c \leftarrow \max(c, 13[1 - \text{smoothstep}(8, 17, d)]). \quad (40)$$

Near the harvester, the clearance is raised to at least 12 while its attack time is below 11. Vertical position follows the target at response rate 8 and is never permitted below terrain height plus 3. This is an arcade clearance rule, not exact mesh collision or aerodynamic physics. Aircraft pitch follows $-0.3\|\mathbf{v}_{XZ}\|$ at response rate 6 to visually indicate forward motion.

The aircraft root matrix is $T(\mathbf{p})R_y(\theta)R_x(\text{pitch})R_z(\text{roll})$. Each wing is transformed relative to this root; translation and articulation therefore remain attached during all player movement.

Implementation reference:

src/arrakis_game.h:199--213; src/arrakis.cpp:1529--1533

3.3 Wing and Machinery Animation

The aircraft has eight wings, four on each side. Wing phase advances at 23 radians/second normally, 36 when boosting and 18 on the title screen. The primary flap signal has amplitude 34° :

$$\beta_1 = 34 \sin \phi, \quad \beta_2 = 34 \sin(\phi + 0.75\pi). \quad (41)$$

The second signal is offset by three quarters of π , not exactly opposite phase. A twist term $8 \cos \phi$ side adds pitching variation. Combined hinge rotation and local scaling create a mechanical insect-like motion. Engine flame length changes from 0.6 to 1.4 during boost and its emission increases.

Harvester sprockets, track pads and drum rotation depend on accumulated horizontal travel ℓ , not simply wall-clock time. Representative phases are 80ℓ degrees for sprocket markers, $2\pi j/10 + 0.7\ell$ for track pads and 70ℓ degrees for the intake drum. Machinery therefore stops when the harvester stops rather than appearing to run independently of its motion.

Implementation reference:

`src/arrakis.cpp:1066--1198,1255--1274; src/arrakis_game.h:213`

3.4 Harvester Route and Terrain Alignment

At route time t , the harvester follows

$$x(t) = -40 + 0.62t + 15 \sin(0.045t), \quad (42)$$

$$z(t) = -55 + 22 \sin(0.032t) + 9 \sin(0.071t), \quad (43)$$

$$y(t) = H(x(t), z(t)). \quad (44)$$

Its forward direction follows the tangent, using

$$x'(t) = 0.62 + 0.675 \cos(0.045t), \quad (45)$$

$$z'(t) = 0.704 \cos(0.032t) + 0.639 \cos(0.071t), \quad (46)$$

$$\psi = \text{atan2}(-x', -z'). \quad (47)$$

The heading is converted to degrees for the stored yaw. Terrain samples along its forward/right axes provide pitch and roll. The route time is clamped to the breach deadline B , so its position and yaw freeze when the attack begins.

For attack time $a = t_{\text{mission}} - B$, the vehicle sinks by

$$h_{\text{sink}} = 20 \text{smoothstep}(6, 12, a). \quad (48)$$

An added X tilt of $0.8 \text{smoothstep}(6, 12, a)$ radians accompanies the descent. It is no longer drawn at $a \geq 12$, conveying that it has been consumed. The three spice tanks are also hidden after this interval; they do not move or deploy independently.

Implementation reference:

`src/arrakis_game.h:44--67; src/arrakis.cpp:1535--1566`

3.5 Worker Evacuation and Rescue States

Group j releases at $t_j = 6 + 7j$ seconds. The harvester position and heading used for that group's exit are sampled at its scheduled release time, not the first frame after release. This avoids destination drift between frame rates.

With harvester position $\mathbf{o}$, forward $\mathbf{f}$ and right $\mathbf{r}$, each group uses

$$\mathbf{e} = \mathbf{o} + 6.1\mathbf{r} - 1.2\mathbf{f}, \quad (49)$$

$$\mathbf{c} = \mathbf{e} + 14\mathbf{r}, \quad (50)$$

$$\mathbf{d}_s = \mathbf{c} + 1.7 \left(s - \frac{n-1}{2} \right) \mathbf{f} + 1.3(s \bmod 2)\mathbf{r}. \quad (51)$$

Here n is group size and s is the member's slot. These offsets spread people around a fixed group marker. The final group releases at 104 seconds. Workers run horizontally toward their destination at 3.8 units/second, sample terrain height at each update, and change from Running to Waiting on arrival.

Running limb stride is $24 \sin(11t_{\text{anim}} + i)$ degrees; waiting stride is $4 \sin(2t_{\text{anim}} + i)$, giving idle movement. Amber rings/poles remain only while a released group still has visible running/waiting members. Individual inside, aboard, safe and lost workers are tracked logically but not drawn in the world.

Implementation reference:

src/arrakis_game.h:141--154,215--250; src/arrakis.cpp:1440--1508

3.5.1 Pickup and delivery rules

Pickup requires a running/waiting worker at horizontal distance *less than* 12, an available seat, Space held, actual terrain clearance ≤ 11 and horizontal speed < 6 . Staying eligible for about 0.42 seconds completes one pickup; changing target or failing the conditions resets progress. During pickup, smoothstep progress lifts the target visually and a cyan line connects the aircraft and worker.

Delivery requires base distance < 14 , passengers aboard, Space held and the same low/steady conditions. A continuous 1.2-second dwell unloads the whole cabin, marks those people *Safe* and updates secured/best counts. Leaving the conditions resets the dwell. An eight-seat cabin requires multiple round trips for all 36 people.

The state progression is

Inside $\longrightarrow$ Running $\longrightarrow$ Waiting $\longrightarrow$ Aboard $\longrightarrow$ Safe.

Unresolved crew can instead become *Lost* through hazards or timeout. A running person may be picked up before reaching the waiting state.

Implementation reference:

src/arrakis_game.h:261--305; src/arrakis.cpp:1490--1508

3.6 Worm Pursuit, Attack and Scene Escalation

The worm-to-harvester gap is

$$g(t) = 220 - 202 \text{ clamp}(t/B, 0, 1). \quad (52)$$

The worm is positioned at the current harvester location minus $(g, 0, 0)$, with its own terrain-sampled height and fixed yaw 90° . It therefore approaches from world west, reducing its gap from 220 to 18, rather than pathfinding along the harvester's actual trail. The visual worm pose adds the detailed rise/attack/retreat motion described in Section 2.

The mission uses the following separate hazard rules; they do not depend on triangle-level mouth collision.

Stage	Crew danger	Aircraft danger
Approach: $a < 0$, elapsed > 8	Running/waiting crew within 16 horizontal units of worm centre.	Distance < 18 and aircraft below worm terrain height +22.
Breach	For $3 < a < 12$, outside crew within 32 horizontal units.	For $3 < a < 15$, distance < 24 and aircraft below worm terrain height +40.

Stage	Crew danger	Aircraft danger
Swallow	For $12 \leq a < 18$, outside crew within 44 units; all crew still inside the harvester are lost at $a \geq 12$.	Uses the breach condition until $a = 15$; there is no additional swallowing envelope.
Final extraction	Outside crew are not threatened by the worm from $a \geq 18$; rescue continues until $a = 60$.	Safe flight and rescue remain available unless the aircraft was already lost.

An aircraft collision marks every crew member not already safe/lost as lost, not only the cabin occupants, and ends the mission. Previously secured workers remain saved. At the 60-second extraction timeout, unresolved and aboard workers are lost. A mission can end earlier when all crew have been accounted for and none remain aboard.

These rules make the scene dynamic at several levels: the player changes position and rescue state, independent workers and machinery move, a threat approaches, the harvester is removed, environmental dust changes source, the timer switches purpose, and debrief becomes available. This is the distinction between *dynamic objects* and a *dynamic scene*.

Implementation reference:

src/arrakis_game.h:68--78,251--258,298--305

3.7 Camera Movement

All three camera modes are third-person, heading-following views. The close preset is called “Cockpit/Close” in code but is not a cockpit interior. For vertical mouse signal $D(u_y)$ and arrow-controlled offset p_o ,

$$p = \text{clamp}(12 - 22D(u_y) + p_o, 6, 65) \text{ degrees}, \quad \mathbf{b} = -\mathbf{f}. \quad (53)$$

The offsets from the following target are

$$\mathbf{o}_{\text{wide}} = 40 \cos p \mathbf{b} + (0, 40 \sin p, 0), \quad (54)$$

$$\mathbf{o}_{\text{close}} = 20 \cos p \mathbf{b} + (0, 20 \sin p + 2.2, 0), \quad (55)$$

$$\mathbf{o}_{\text{tactical}} = 26\mathbf{b} + (0, 100, 0). \quad (56)$$

The target follows the aircraft at rate 12; eye height smooths at rate 9. Horizontal camera placement uses the heading offset directly, keeping screen direction aligned with aircraft heading. The camera is kept above the terrain, and world-up is $(0, 1, 0)$ so banking does not roll the view. Arrow elevation changes at $40^\circ/\text{second}$ and its offset is bounded to $[-10^\circ, 45^\circ]$.

The view matrix is a look-at matrix from camera eye to target. The perspective projection uses vertical field of view 58° , current framebuffer aspect ratio, near plane 0.4 and far plane 1200. Camera reset/resume/resize/fullscreen handling centres the cursor to prevent an unintended sharp turn.

Implementation reference:

src/arrakis_game.h:107--118; src/arrakis.cpp:1903--1961

3.8 Particles as Dynamic Environmental Objects

The desert has 2,400 dust particles. Wind direction is the normalized vector $(-0.92, -0.04, 0.38)$ and base wind speed is 18. A gust multiplier $1 + 0.35 \sin(1.8t_{\text{wall}})$ changes intensity. Particle lifetime

decreases each update and position is advected by its current velocity; particles that expire or leave the active region respawn.

Respawn sources depend on particle index and mission stage: harvester dust, worm dust, aircraft downwash when Space is held, or ambient upwind sand. Respawn lifetimes lie between 2 and 3.98 seconds. A ground contact places a particle at $\text{terrain} + 0.2$ and gives upward velocity $|v_y| \cdot 0.5 + 0.8$. This produces drifting clouds and local activity without a fluid simulator.

Initial and update random generators use fixed seeds 1337 and 1984. However, gusts use GLFW wall time, and particle updates run once per rendered frame, so visual dust is not guaranteed identical across all runs. It does not push the aircraft or deform the terrain.

Implementation reference:

`src/arrakis.cpp:153--255`

4 Rendering, Appearance and Additional Features

4.1 Forward Rendering Pipeline

The scene uses a forward renderer: each visible object's fragment shader calculates its material/lighting directly, rather than filling a deferred G-buffer. Per frame, the graphics stages are

Shadow depth → **Sky** → **Opaque scene** → **Transparent dust** → **HUD**.

The shadow target is the only separate framebuffer. Main colour rendering goes directly to framebuffer 0. Four-sample MSAA is requested at context creation; availability depends on the driver. Depth testing uses `LEQUAL`, back-face culling is normally enabled, and individual two-sided wings temporarily disable culling.

Scene positions are transformed by *PVM*; normals use the inverse-transpose normal matrix so nonuniformly scaled ellipsoids/cylinders can be shaded correctly. World position and normal pass to the fragment shader along with light-space coordinates. Identical scene assemblies are traversed for the shadow and visible passes. Repeated vehicle parts are individual draws, while dust uses instancing and all worm teeth are combined into one mesh.

Implementation reference:

`src/arrakis.cpp:640--676,1778--1782,1957--2110`

4.2 Material Design

Each material supplies diffuse colour, specular colour, shininess exponent, emission and a surface-mode flag. Mode 0 is ordinary material, mode 1 sand, and mode 2 rock strata. There are no metalness, roughness, alpha or imported normal-map fields.

Material group	Representative diffuse RGB	Design purpose
Terrain	(0.76, 0.49, 0.23)	Warm sand, low-exponent specular, slope/ripple/grain detail and wrapped diffuse lighting.
Aircraft hull/dark parts	(0.22, 0.23, 0.22) / (0.13, 0.14, 0.13)	Dark mechanical silhouette with moderate highlights.
Canopy/wing	(0.06, 0.08, 0.11) / (0.18, 0.19, 0.18)	High shininess 128/90; canopy specular 0.9 and wing specular 0.7. Both are opaque.
Exhaust	(1, 0.55, 0.12)	Orange emission increases under boost.
Harvester chassis/treads/rust	(0.42, 0.34, 0.25) / (0.11, 0.11, 0.10) / (0.48, 0.24, 0.12)	Separate body, dark tracks and weathered machinery.
Spice scoop/vents	(0.96, 0.42, 0.04)	Bright orange emission distinguishes spice-related fittings.
Bridge glass/lights	(0.1, 0.2, 0.25) / (1, 0.95, 0.7)	Glossy bridge viewport and pale emissive ornaments; no real spotlight.

Material group	Representative diffuse RGB	Design purpose
Tank cradle/vessel/seams	(0.24, 0.23, 0.22) / (0.52, 0.46, 0.38) / (0.18, 0.17, 0.16)	Contrasting support, metal vessel and reinforcing bands.
Rock strata	(0.42, 0.28, 0.17) / (0.32, 0.20, 0.11)	Two earthy bands with procedural grain.
Cyan/amber markers	(0.05, 0.5, 0.55) / (0.9, 0.5, 0.1)	Emissive colour coding separates safe base from evacuation groups.
Worker cloth/visor	(0.30, 0.24, 0.17) / (0.08, 0.20, 0.27)	Matte clothing and glossy visor for small readable characters.
Worm skin/rim	(0.43, 0.27, 0.12) / (0.49, 0.31, 0.14)	Low-shininess organic brown surface.
Funnel/throat/teeth	(0.15, 0.075, 0.029) / (0.009, 0.004, 0.002) / (0.58, 0.43, 0.24)	Recess darkens toward the throat; lighter teeth stand out against the cavity.

Emission adds a term to the emitting surface's colour. It does not cast light on nearby objects and does not imply bloom. These distinctions explain why markers and vents look bright without a point-light system.

Implementation reference:

src/arrakis.cpp:972--978,1051--1079,1214--1235,1321--1335,1389--1426,1520--1524; src/arrakis_worm.h:317--332

4.3 Lighting Equation

One constant directional sun uses

$$L = \text{normalize}(-0.35, 0.12, -1), \quad C_{\text{sun}} = (1.15, 1.04, 0.85), \quad C_{\text{ambient}} = (0.34, 0.38, 0.44). \quad (57)$$

Its low elevation creates long desert shadows. For normal N , viewing direction V and halfway vector $H = \text{normalize}(L + V)$, ordinary diffuse and Blinn–Phong specular are

$$d = \max(N \cdot L, 0), \quad s = \max(N \cdot H, 0)^n. \quad (58)$$

Here n is material shininess. Sand instead uses a softer wrapped diffuse response

$$d_{\text{sand}} = \max\left(\frac{N \cdot L + 0.35}{1.35}, 0\right). \quad (59)$$

This keeps dune slopes from becoming excessively dark. With shadow fraction S , the lit RGB is

$$C_{\text{lit}} = C_{\text{ambient}}C_{\text{base}} + (1 - S)[dC_{\text{sun}}C_{\text{base}} + sC_{\text{spec}}C_{\text{sun}}] + C_{\text{emission}}. \quad (60)$$

RGB products are componentwise. The shader is an efficient stylized lighting model, not a physically based metal/roughness BRDF. It has no point/spot lights, global illumination, environment-map lighting or ambient occlusion.

Implementation reference:

src/arrakis.cpp:713--762,1835--1840

4.4 Procedural Textures and Surface Detail

4.4.1 Sand ripples, slope tint and sparkle

No photographic or bitmap texture is loaded. Sand variation is calculated from world position:

$$r_1 = 0.5 \sin(2.2x + 1.4z) + 0.5, \quad r_2 = 0.5 \sin(0.8x - 1.9z) + 0.5, \quad r = (r_1 + r_2)/2. \quad (61)$$

Diffuse colour receives a slope multiplier $0.82 + 0.33 \text{ clamp}(N_y, 0, 1)$ and a small ripple offset $(0.035, 0.02, 0.008)(r - 0.5)f$, where

$$f = 1 - \text{smoothstep}(25, 120, \|\text{viewPos} - P\|). \quad (62)$$

Thus fine details fade in the distance, reducing distracting noise. A normal perturbation creates a bump-like ripple effect without displacing geometry:

$$N \leftarrow \text{normalize}[N + (0.055 \cos q, 0, 0.035 \cos q)f], \quad q = 2.2x + 1.4z. \quad (63)$$

Sparse sparkle is selected by the procedural hash

$$h = \text{fract}[\sin(12.9898x + 78.233z) 43758.5453]. \quad (64)$$

For $h > 0.88$, a sharp $\max(N \cdot H, 0)^{120}$ term adds a small warm highlight, also scaled by f . The ripples and sparkle are appearance effects, not extra terrain vertices or physically individual sand grains.

Implementation reference:

`src/arrakis.cpp:726--745`

4.4.2 Rock strata and grain

Rock-mode colour is multiplied by $0.88 + 0.12l + 0.07g$, with

$$l = \sin(5y + \sin(0.8x)), \quad g = \text{fract}[\sin(P \cdot (12.3, 42.8, 27.1)) 43758.5453]. \quad (65)$$

The height-based bands suggest strata, while the hash provides small grain. Flat face normals and geometric perturbation give the rocks their faceted shape; the shader adds detail without additional meshes. Although the vertex format carries UVs, these patterns are world-coordinate functions rather than image sampling.

Implementation reference:

`src/arrakis.cpp:720--724`

4.5 Shadows: Depth Map, Bias and Filtering

4.5.1 Depth pass

A 2048×2048 depth-only texture/framebuffer stores the nearest light-space surface. It uses nearest filtering, a depth-one border and no colour attachments. Framebuffer completeness is checked during initialization. Upload type `GL_FLOAT` does not by itself guarantee a 32-bit floating-point internal depth format.

The sun view is centred on the ground below the aircraft, with light eye at that point plus $120L$. The orthographic volume is $[-90, 90]$ in horizontal/vertical light coordinates and near/far $2/320$. It covers a moving 180-unit region, not the entire 900-unit desert. All opaque scene parts are rendered with polygon offset $(2.5, 4.0)$ to reduce self-shadow artefacts.

4.5.2 Shadow comparison

A visible fragment's light coordinates are divided by w and remapped to $[0, 1]$. Outside-map fragments are unshadowed. Bias is

$$b = \max[0.0035(1 - N \cdot L), 0.0008]. \quad (66)$$

The shader also estimates the receiver's depth slope from screen derivatives of projected light coordinates. Writing these derivatives as d_x, d_y , define

$$D = d_{x,x}d_{y,y} - d_{x,y}d_{y,x}, \quad (67)$$

$$S_p = \frac{(d_{x,z}d_{y,y} - d_{y,z}d_{x,y}, d_{x,x}d_{y,z} - d_{y,x}d_{x,z})}{D}. \quad (68)$$

If $|D| \leq 10^{-9}$, the slope is set to zero. For sample offset $o = (i, j)/\text{textureSize}$, comparison tests

$$\text{projectedDepth} + S_p \cdot o - b > \text{storedDepth}. \quad (69)$$

Twenty-five comparisons at $i, j \in \{-2, -1, 0, 1, 2\}$ are averaged. This fixed 5×5 percentage-closer filter softens edges and the slope correction reduces surface acne. It is not contact-hardening or cascaded shadow mapping. The moving finite coverage can still show resolution/boundary artefacts, and there is no texel-snapped stabilization.

Implementation reference:

src/arrakis.cpp:687--711,923--950,1981--2006

4.6 GPU Terrain Collapse During the Attack

The attack visually lowers terrain around the worm in both colour and shadow vertex shaders. With horizontal distance d_w from worm centre and attack time a ,

$$\Delta y = 13 \text{ smoothstep}(7, 14, a)[1 - \text{smoothstep}(5, 48, d_w)]. \quad (70)$$

The vertex height is reduced by Δy , reaching up to 13 units near the centre and tapering to zero at radius 48. It strengthens the swallowing scene and keeps shadow geometry consistent with visible geometry.

This deformation is **visual and GPU-only**. CPU height queries, object placement, aircraft/camera clearance, particle-ground checks and original terrain normals are not changed. Therefore it is not persistent terrain destruction or a collision-accurate crater; objects and lighting can disagree with the depressed surface. This limitation is important when explaining the feature honestly.

Implementation reference:

src/arrakis.cpp:648--658,783--786,1996--2000,2055--2057

4.7 Procedural Sky and Sun

An oversized full-screen triangle reconstructs a world-view ray from inverse view-projection. It is a generated background, not an image cubemap/skybox. For upward ray component $e = \text{clamp}(r_y, 0, 1)$, the shader first blends dusty horizon (0.90, 0.74, 0.47) to low blue (0.48, 0.69, 0.88) over $\text{smoothstep}(0, 0.13, e)$, then blends toward high blue (0.10, 0.34, 0.72) with $e^{0.65}$.

For $u = \max(r \cdot L, 0)$, sun contributions are

$$D_{\text{sun}} = 1.9 \text{ smoothstep}(\cos 0.80^\circ, \cos 0.58^\circ, u), \quad (71)$$

$$C_{\text{corona}} = 0.30u^{180}, \quad C_{\text{haze}} = 0.10u^{12}. \quad (72)$$

Their sum times sun colour is added to the sky. Depth writes are disabled so the background does not occlude objects. There are no clouds, moons/stars, physically integrated atmospheric scattering or day/night cycle.

Implementation reference:

src/arrakis.cpp:833--876,952--965,2018--2033

4.8 Tone Mapping and Distance Fog

The surface shader applies a compact colour compression/gamma-like mapping

$$C_{\text{mapped}} = \left(\frac{C_{\text{lit}}}{C_{\text{lit}} + 0.8} \right)^{1/1.8}. \quad (73)$$

Then distance $d = \|\text{viewPos} - P\|$ controls fog:

$$F = 1 - e^{-(0.0022d)^2}, \quad C_{\text{out}} = (1 - F)C_{\text{mapped}} + F(0.90, 0.74, 0.47). \quad (74)$$

Distant geometry approaches the warm horizon colour, producing a dusty sense of scale. This is analytic distance fog, not volumetric/height fog. Tone mapping occurs per scene fragment; there is no separate HDR colour target, bloom extraction, blur chain or full-screen post-processing compositor. The sky/dust/HUD have their own simpler shaders rather than sharing this surface calculation.

Implementation reference:

src/arrakis.cpp:764--770

4.9 Instanced Transparent Dust

Each dust particle is a quad oriented with the camera's right/up vectors:

$$P_{\text{world}} = P_{\text{centre}} + \text{camRight}(x_{\text{quad}s}) + \text{camUp}(y_{\text{quad}s}). \quad (75)$$

The fragment shader discards points outside a UV-centred radius 0.5. A soft circular edge uses the mask $1 - \text{smoothstep}(0, 0.5, r)$. Lifetime alpha fades in/out using

$$f = \text{clamp}(\text{life}/0.5, 0, 1) \text{ clamp}((\text{maxLife} - \text{life})/0.3, 0, 1). \quad (76)$$

Particles are sorted far-to-near by squared camera distance, streamed as instance data and drawn in one instanced indexed call. Alpha blending uses source-alpha/one-minus-source-alpha; depth testing stays enabled, while depth writes/culling are disabled. This lets dust blend over the scene without blocking later dust through opaque depth writes.

The pool's motion and emitters are described in Section 3. The dust shader does not calculate scene lighting, shadows or its own fog. "Soft" refers to the alpha edge, not depth-aware soft-particle intersection fading. This is a visual dust system, not smoke/fluid physics. Instancing reduces draw overhead for 2,400 particles, unlike the individually submitted vehicle parts.

Implementation reference:

src/arrakis.cpp:794--831,1810--1821,2066--2110

4.10 Inspection and Robustness Features

Wireframe helps inspect triangle topology; the HUD is explicitly kept filled and readable. Fullscreen and resized framebuffer aspect ratios are supported. The HUD saves/restores relevant OpenGL state, uses screen-space alpha blending and is drawn after all world effects. Normal exit explicitly frees buffers, vertex arrays, shader programs, shadow resources, HUD/sky/particles and the worm meshes.

Shader compile/link and shadow-framebuffer checks print diagnostics. Graphical smoke tests inspect OpenGL errors. These improve demonstration reliability but do not constitute exhaustive device compatibility testing. Because wireframe is a global polygon mode, it can also affect sky/dust and the shadow pass; it is an inspection mode rather than an alternate polished rendering pipeline.

Implementation reference:

src/arrakis.cpp:881--950,1623--1626,2158--2200; src/arrakis_hud.h:68--107,512--549;
src/arrakis_worm.h:260--265

5 Complete Project Pipeline

5.1 Source Organization and Responsibilities

File or directory	Role
src/arrakis.cpp	Main application; terrain/mesh helpers; shaders/materials; particles; model assemblies; shadow and scene passes; input, camera and timing integration; smoke-test capture; cleanup.
src/arrakis_game.h	Mission data, aircraft motion, camera offsets, route, releases, hazards, winch/delivery and scoring.
src/arrakis_worm.h	Body, mouth, teeth, throat and lip mesh construction; worm animation pose and drawing.
src/arrakis_hud.h	Bitmap glyphs, overlay geometry/shader, layout, radar, counters, timers and menus.
src/arrakis_audio.h	Sound search/loading, miniaudio engine, positional mixing, event detection, mute and shutdown.
tests/arrakis_tests.h	Headless test runner and 27 mission/control cases.
third_party/	Generated GLAD OpenGL loader and miniaudio implementation. These are third-party support libraries, not original project algorithms.
assets/audio/	Eight runtime sound effects and retained license notice.
CMakeLists.txt, Arrakis_001.vcxproj, vcpkg.json	Primary CMake build, alternative Visual Studio project and optional GLM dependency manifest.
docs/, tools/	Existing explanation chapters, proposal/reference images, generated source-reference PDF/manifest and Python documentation generator. Reference images are not loaded as scene assets.
report/	This independent LaTeX report, real screenshot figures, capture/verification scripts and compiled submission PDF.

The application compiles the main C++ file, GLAD C loader and miniaudio implementation. The project headers are included in a dependency-aware order into the main translation unit; they are not separately linked gameplay plugins. GLM supplies vector/matrix operations, GLFW provides the window and events, GLAD loads graphics entry points, and miniaudio handles playback.

5.2 From Source to a Running Frame

1. **Configure and compile:** CMake locates headers/libraries, compiles C++17 sources, links OpenGL/GLFW and copies audio assets beside the executable.
2. **Choose launch mode:** `--test-game` runs headless tests before any window/audio initialization. A graphical smoke flag selects a reproducible rendering setup; ordinary launch starts the title screen.
3. **Create the graphics context:** GLFW requests OpenGL 3.3 core, a 1280×720 window, four-sample MSAA and VSync. GLAD resolves OpenGL functions. Startup prints graphics device, API version and resolution.

4. **Initialize resources:** Generate terrain/basic shapes/worm meshes; create VAOs/VBOs/index buffers; compile embedded GLSL shaders; allocate shadow, sky, particle and HUD resources; initialize audio and mission state.
5. **Receive input:** GLFW callbacks store held keys and normalized mouse position, process mode toggles and handle resize/focus/cursor events.
6. **Update simulation:** Measure/clamp time, step flight and mission logic, update the harvester and crew, apply hazards/interactions and refresh presentation particles/animation when not paused.
7. **Update camera and audio:** Compute safe following view/projection, update listener/source positions, continuous sound parameters and one-shot events.
8. **Render:** Draw shadow depth, sky, lit opaque objects, sorted instanced dust and filled HUD. Swap buffers and process events for the next frame.
9. **Shut down:** Release meshes/buffers/programs/shadow/HUD/particle resources and audio, destroy the window and terminate GLFW.

No asset-import or offline mesh-conversion stage is needed: source functions generate all graphics at startup. Shader source is embedded, so shader compilation happens on the graphics driver at runtime, with errors printed if compilation/linking fails.

Implementation reference:

src/arrakis.cpp:1697--2210; CMakeLists.txt:1--56

5.3 Complete User Controls

Input	Action
Enter	Start from title or advance from debrief to the next wave. During active flight it does not restart the mission.
Mouse hover	Horizontal offset turns aircraft/camera; vertical offset changes following-camera elevation. No mouse button is required. Centre dead zone stops new turn input.
W / S	Heading-relative forward/reverse thrust.
A / D	Heading-relative left/right strafe, with heading unchanged.
Q / E	Increase/decrease requested terrain-following height.
Space	Brake and descend toward rescue height; winch eligible nearby crew or unload over the cyan pad.
Left Shift	Rechargeable movement boost; release after depletion to unlock it.
C	Cycle wide, close and tactical camera presets.
Left / Right arrows	Turn, as a backup or addition to mouse steering.
Up / Down arrows	Adjust following-camera elevation offset.
P	Pause/resume an active mission.
M	Master mute/unmute.
R	Retry/reset the current mission wave.
F	Toggle filled versus wireframe scene rendering. The HUD remains filled.

Input	Action
F11	Toggle primary-monitor fullscreen; return to a 1280×720 window.
Esc	Close the application.

Window-focus loss clears held input, mouse axes and turn inertia and pauses an active mission. Focus regain does not automatically resume it. Mouse enter/leave clears steering, while resize, fullscreen and mission/resume operations centre the cursor. Minimized windows wait for events rather than repeatedly drawing. There are no scroll, gamepad, mouse-button, weapon or tank-deployment controls.

Implementation reference:

`src/arrakis.cpp:1584--1661,1743--1760,1857--1862`

5.4 HUD and Radar Implementation

The HUD is built with OpenGL instead of an external interface framework. A built-in 5×7 bitmap font converts active glyph cells into coloured rectangle triangles. Strings are uppercased for the supported glyph set; unknown glyphs use a question-mark pattern. This avoids a runtime font texture or font-file dependency.

The interface exposes all important mission state: secured/36, aboard/8 with cabin indicators, inside/outside/lost crew, released groups, mission wave, breach/extraction timer, actual altitude, horizontal speed, smoothed FPS, boost resource, mute status, pursuit bar and interaction progress. Context messages explain excessive height, a full cabin, winching, unloading, harvester loss and aircraft loss. At 15 seconds or less to breach, the warning becomes more urgent. Title, pause and debrief use dimmed panels with relevant instructions.

The north-up radar maps world X, Z in $[-300, 300]$ to a small screen rectangle. It shows the base as a cyan outlined symbol, outside crew amber, harvester white, worm red and aircraft as a cyan heading-oriented triangle. It does not rotate with the camera. The label “worm range” displays the worm-to-harvester gap, not aircraft-to-worm distance, a useful distinction when interpreting the telemetry.

Four centre guide brackets show the mouse dead-zone corners during active unpaused flight. Layout adapts to framebuffer size through compact/short modes, wrapping and limited action text; radar or telemetry may be omitted if space is insufficient. Text and panels are rendered in screen space with scene wireframe disabled for this pass.

Implementation reference:

`src/arrakis_hud.h:149--222,257--451,519--523; src/arrakis.cpp:2130--2160`

5.5 Audio Pipeline and Event Design

The miniaudio engine loads the eight effects below. Sources use inverse-distance attenuation with effect-specific minimum/maximum ranges, while the listener follows camera eye/look direction with world-up Y . Vehicle loops, distress calls and the worm are therefore located in the 3D scene rather than played as a single undifferentiated soundtrack.

Effect	Playback	Purpose and behaviour
<code>ornithopter_flight.mp3</code>	Loop	Aircraft engine/wing ambience; pitch follows speed and volume changes with boost.

Effect	Playback	Purpose and behaviour
ornithopter_boost.mp3	Loop	Fades toward audible volume during active boosting, fades out otherwise.
worm_rumble.mp3	Loop	Positional threat sound; volume strengthens as the harvester gap decreases.
worm_breach.mp3	One-shot	Starts once near the breach transition.
harvester_engine.wav	Loop	Moving machinery sound while flying and before attack time 11.
rescue_winch.wav	One-shot	Pickup progress/cabin increase feedback.
rescue_safe.wav	One-shot	Confirms secured-score increase on delivery.
crew_help.wav	One-shot	Periodic nearby distress calls.

Aircraft pitch target is $0.92 + \text{clamp}(v/42, 0, 0.38)$, smoothed at rate 6. Boost volume approaches 0.90 at rate 8 while boosting. Worm rumble varies using $\text{mix}(1.2, 0.8, \text{clamp}(g/200, 0, 1))$. Events are detected from state changes, such as increased cabin count or increased secured count, instead of replaying a sound every render frame.

The configured attenuation ranges are 12–150 for aircraft flight/boost, 30–320 for rumble, 45–380 for breach, 18–200 for the harvester, 15–120 for winching, 20–140 for delivery and 16–85 for crew calls. A nearby call timer initially waits three seconds, then uses 5.5–8.4 seconds between qualifying calls; otherwise it checks again after two seconds. The update's nearest-crew fallback is the harvester position when no outside crew exist, so this audio helper is a feedback approximation rather than a complete dialogue/voice-state system.

Audio searches `assets/audio/` under the current working directory and up to four ancestor levels. Run from the project root or executable directory. This is not executable-relative resource lookup for an arbitrary launch directory. Device initialization failure leaves the game playable in silence; a missing individual effect similarly does not terminate rendering.

Mute/pause set master volume to zero rather than suspending underlying playback. A one-shot may finish while inaudible. Flight ambience can continue on title/debrief; the harvester/rumble are mission-phase gated. There is no music player, voice-dialogue system or device-selection menu. The existing Kenney notice is retained, but it should not be treated as per-file proof of the origin of every unrelated sound.

Implementation reference:

`src/arrakis_audio.h:11--25,62--126,158--276; src/arrakis.cpp:1963--1971`

5.6 Build, Dependencies and Distribution

The primary build requires CMake 3.20+, a C++17 compiler, OpenGL 3.3-capable graphics, GLFW, matching GLAD headers and GLM. This installation uses local discovery hints under `E:/glfw_nec` and `E:/glm`; the libraries are not all bundled as complete portable dependencies. The vendored GLAD implementation was generated for GL 4.6 core, so external headers must match that loader even though the application requests only OpenGL 3.3 functionality.

```
cmake -S . -B build/rescue -G "Visual Studio 17 2022" -A x64
cmake --build build/rescue --config Release
.\build\rescue\Release\Arrakis.exe
```

On another machine, set the dependency locations during CMake configuration:

- `GLAD_INCLUDE_DIR`: matching GLAD headers.

- `GLFW_INCLUDE_DIR`: GLFW headers.
- `GLM_INCLUDE_DIR`: GLM headers.
- `GLFW_LIBRARY`: the GLFW library binary.

The optional `vcpkg` manifest installs GLM only, not GLFW or GLAD. The build definition `GLFW_INCLUDE_NONE` prevents GLFW from including conflicting OpenGL headers. CMake links GLFW/OpenGL, compiles the loader/audio implementation and copies sound assets post-build. The alternative Visual Studio project uses v143/Windows SDK and laboratory-specific x64 paths; Win32 configurations do not have equivalent complete dependency setup.

Distribute the Release executable with its copied `assets/audio/` directory. There is no installer or packaging target. The graphics are generated at startup, so there is no model/texture folder to ship. Verify the driver and library linkage on the intended demonstration machine rather than assuming an executable built on one computer runs identically everywhere.

Implementation reference:

`CMakeLists.txt:1--56`; `Arrakis_001.vcxproj`; `vcpkg.json`; `README.md:64--85`

5.7 Testing and Observed Results

Verification for this report used the current Release build on Windows with Intel Iris Xe graphics and an OpenGL 3.3 context at 1280×720. These are actual command results, not inferred passes.

Verification	Observed result
Release build	CMake/MSBuild completed successfully.
CTest	RescueMissionLogic passed; 1/1 registered tests passed.
<code>--test-game</code>	27/27 cases, 1,057 checks, zero failures.
<code>--smoke-test</code>	Passed 120 rendered frames; scene screenshot captured.
<code>--smoke-breach</code>	Passed 120 rendered frames; breach screenshot captured.
<code>--smoke-mouse</code>	Passed 120 frames and heading/camera alignment checks; tactical-control screenshot captured.
<code>--smoke-pursuit</code>	Passed 120 rendered frames; pursuit screenshot captured.

The headless cases cover reset/title and pause, scheduled release and fixed destinations, pickup and cabin capacity, height/speed restrictions, base-only delivery, swallowing, carried survivors after breach, collision and saved score, timeout accounting, retry, wave limits, full rescue state flow, analytic routes, hazard boundaries, cursor normalization/invalid coordinates, mouse/arrows, yaw-relative translation, damped steering, frame-rate comparisons and all camera offsets/pitch limits.

One input-only test completed five rescue flights, saved all 36 with zero losses in about 110.445 simulated seconds, used real boost and maintained minimum clearance 3. It uses a scripted controller driving keyboard-equivalent inputs; it is not a recording of a human player. Other mechanics tests deliberately place state/positions to isolate conditions. Tests reconcile all 36 workers and the eight-seat limit.

Smoke tests inspect OpenGL errors, not golden-image similarity. They run 120 frames at a fixed simulation step and optionally save the last frame as a 32-bit BMP. Mouse smoke also changes camera every 40 frames and checks alignment. These modes require a graphics desktop and are separate from the one headless CTest registration. Passing them confirms the exercised paths, not every possible GPU, window mode or human interaction.

```
ctest --test-dir build/rescue -C Release --output-on-failure
```

```
.\build\rescue\Release\Arrakis.exe --test-game  
.\build\rescue\Release\Arrakis.exe --smoke-test  
.\build\rescue\Release\Arrakis.exe --smoke-breach  
.\build\rescue\Release\Arrakis.exe --smoke-mouse  
.\build\rescue\Release\Arrakis.exe --smoke-pursuit
```

6 Implementation References

The implementation references throughout this report point to the current project files. Local code is the primary evidence; external references below identify the APIs and concepts used, not copied project features.

- `src/arrakis.cpp`: procedural meshes, materials/shaders, shadow map, particles, object drawing, GLFW callbacks and application loop.
- `src/arrakis_game.h`: deterministic mission state, aircraft/camera updates, evacuation, rescue, pursuit and scoring rules.
- `src/arrakis_worm.h`: detailed sandworm meshes, curved teeth, pose and drawing.
- `src/arrakis_hud.h`: bitmap text, overlay meshes, radar and mission instrumentation.
- `src/arrakis_audio.h`: miniaudio integration, sound loading, playback/mixing and fallback.
- `tests/arrakis_tests.h`, `CMakeLists.txt`, `Arrakis_001.vcxproj`, `vcpkg.json`: verification and build integration.
- `README.md`, `tools/generate_docs.py`, `tools/requirements.txt`: usage and the existing documentation pipeline. Earlier generated documentation can describe an older source snapshot; this report prioritizes current code.
- Khronos OpenGL documentation: <https://www.khronos.org/opengl/>; GLFW documentation: <https://www.glfw.org/docs/latest/>; GLM documentation: <https://glm.g-truc.net/>; miniaudio documentation: <https://miniaud.io/>.
- Audio attribution is retained in `assets/audio/Kenney-License.txt`; vendored library licenses remain with their distributed sources.